Software Architecture and Design Specification

Project: Hospital Management System

Version: 1.0

Authors: Pratham P Mudakavi, Pratham P Shetty, Prathik M Nambiar, Pratik Patil

Date: 04-10-2026

Status: Draft

Revision History

Version	Date	Author	Change Summary
1.0	04-10-2026	Team 1	Initial SAD based on HMS SRS and test baseline.

Approvals

Role	Name	Signature/Date
Project Team	Team 1	
Requirements & Acceptance Owner	Pratik Patil	Pratik Patil
Faculty / Evaluator		

1. Introduction

1.1 Purpose

This document specifies the proposed software architecture and design of the Hospital Management System (HMS). It translates the requirements baseline into a modular design covering user access, patient, doctor, appointment, admission, room, medical record, billing, payment, reporting, validation, and persistent-storage concerns.

1.2 Scope

The HMS is a standalone C/C++ mini-project for structured management of patient, doctor, appointment, admission, room, medical, and billing information. The initial release uses local persistent storage and does not require external healthcare, insurance, payment-gateway, cloud, or network services.

1.3 Audience

Developers, QA/test engineers, hospital reception and administrative users, doctors and authorized medical staff, billing staff, project evaluators, faculty, and maintenance teams.

1.4 Definitions

HMS, SRS, Patient ID, Doctor ID, Appointment ID, CRUD, UI, DB/File Storage, RTM, UAT.

2. Document Overview

2.1 How to use this document

Provides the HMS architectural view, component responsibilities, design rationale, technology and storage decisions, requirement traceability, security considerations, representative interaction flows, module interfaces, error handling, UX considerations, and open issues.

2.2 Related Documents

Hospital Management System Software Requirements Specification (SRS), Hospital Management System Software Test Plan, Requirements Traceability Matrix (RTM), and the suggested C/C++ program structure in the SRS.

3. Architecture

3.1 Goals & Constraints

Goals: modularity, maintainability, reliability, usability, portability, consistent relationships among hospital records, validated input, safe persistence, and basic reporting. Constraints: C/C++ implementation, academic mini-project scope, local persistent storage, Windows/ Linux compatibility, no mandatory external network communication, and controlled access to sensitive information.

3.2 Stakeholders & Concerns

• Administrator — user/doctor/department/room management, reports, backup, and access control.

• Receptionist — patient registration, search, appointments, admissions, and discharge support.

• Doctor — authorized access to patient records and clinical information entry.

• Billing Staff — bill creation, payment recording, balances, and billing reports.

• Developers/QA — modularity, validation, testability, portability, persistence safety, and traceability.

• Project evaluators/faculty — requirement coverage and demonstrable acceptance criteria.

3.3 Component (UML) Diagram

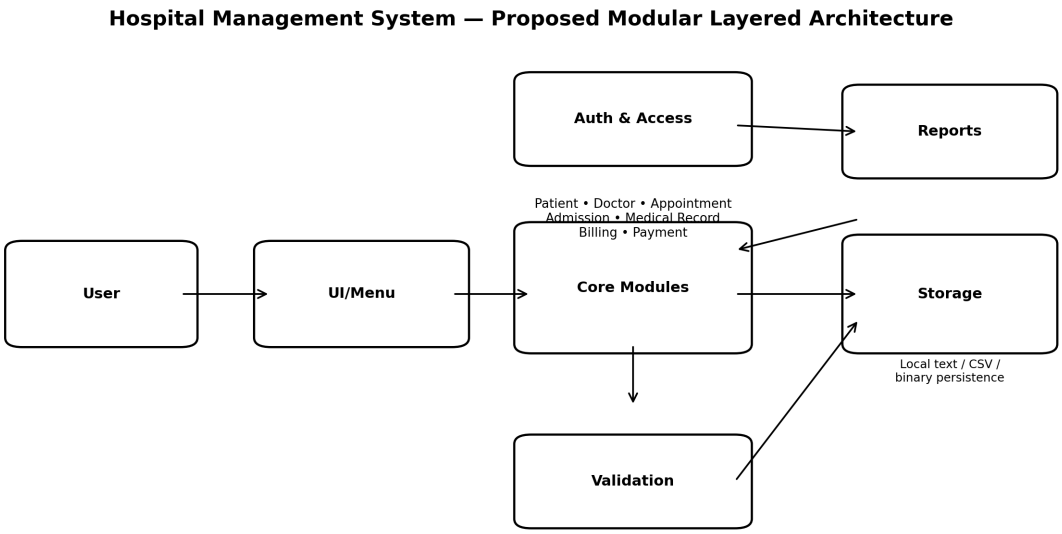


Figure 1. Proposed modular layered architecture derived from the HMS requirements and suggested program structure.

3.4 Component Descriptions

• UI/Menu Layer: presents menus, prompts, forms, confirmations, and reports; the SRS allows a console-based initial interface.

• Authentication & Access: validates authorized users and restricts protected administrative/ medical functions according to role where implemented.

• Patient Management: creates, searches, displays, and updates patient records while preserving unique Patient IDs.

• Doctor Management: stores doctor identity, specialization, department, contact, and availability.

• Appointment Management: creates, modifies, cancels, and displays appointments and validates patient/doctor references.

• Admission & Room Management: records admissions, allocates available rooms/beds, and updates occupancy on discharge.

• Medical Records: stores diagnosis, treatment notes, prescriptions, visit information, and responsible doctor linkage.

• Billing & Payments: creates bills, accumulates charges, records payments, calculates balances, and provides billing summaries.

• Reporting: produces patient, appointment, admission/occupancy, billing, and outstanding-payment summaries.

• Validation: checks mandatory fields, identifiers, dates, numeric values, duplicate records, and other supported constraints.

• Storage: provides local persistence and reload of records across application restarts; the SRS permits text, CSV, binary, or equivalent local files.

3.5 Chosen Architecture Pattern and Rationale

A modular layered architecture is proposed for the HMS. Separation between UI/menu handling, domain modules, validation, reporting, and storage supports the SRS maintainability requirement and keeps each major hospital function independently testable. A distributed microservices architecture is not appropriate for the initial release because the SRS defines a standalone academic application with local persistence and no required network services.

3.6 Technology Stack & Data Stores

Language: C or C++. Compiler: GCC/G++, MinGW, or equivalent. Operating environment: Windows 10/11 or compatible Linux. Storage: local text, CSV, binary, or equivalent persistent files. IDE/editor: any C/C++ compatible environment. No external network communication is required for the initial release.

3.7 Risks & Mitigations

Risk	Mitigation
Inconsistent or corrupted persistent files	Use validation, seeded test data, safe save/exit handling, and backup/export procedures.
Incomplete role/privacy controls	Run negative role tests and restrict sensitive patient/medical information to authorized users.
Invalid or duplicate records	Centralize validation and enforce unique identifiers before persistence.
Limited project time/environment availability	Prioritize high-priority requirements and maintain a reproducible build/test setup.
Requirement or data-format changes	Update the RTM and affected design/test artifacts, then execute focused regression.

3.8 Traceability to Requirements

Requirement	Concern	Design Component
FR-01	User login/access	Authentication & Access
FR-02–FR-04	Patient registration, update, search	Patient Management + Validation
FR-05	Doctor management	Doctor Management
FR-06–FR-07	Appointment scheduling/ modification	Appointment Management + Validation
FR-08–FR-09	Admission and room allocation	Admission & Room Management
FR-10–FR-11	Medical records and discharge	Medical Records + Admission
FR-12–FR-13	Billing and payments	Billing & Payments
FR-14	Reports	Reporting
FR-15	Data validation	Validation
FR-16–FR-18	Persistence, backup, safe exit	Storage + UI/Menu

3.9 Security Architecture

Security is treated primarily as role-based access and information protection within the standalone application. Login/authentication should protect restricted functions; administrator, receptionist, doctor, and billing operations should expose only permitted

functions and data; passwords, if stored, should not be displayed in plain text; sensitive patient and medical information should not be unnecessarily shown to unauthorized roles; and malformed, oversized, empty, duplicate, and boundary inputs should be rejected safely. The initial release does not require external network communication.

4. Design

4.1 Design Overview

The design decomposes the HMS into logical modules corresponding to the SRS entities and workflows. Each module owns its core operations while validation and storage provide cross-cutting services. This supports unit testing of functions, integration testing across modules, and system-level execution of complete user workflows.

4.2 UML Sequence Diagrams

The following representative flows are derived from UC-01 Register Patient and UC-05 Generate Bill. They illustrate the intended interaction between user roles, modules, validation, persistence, and results.

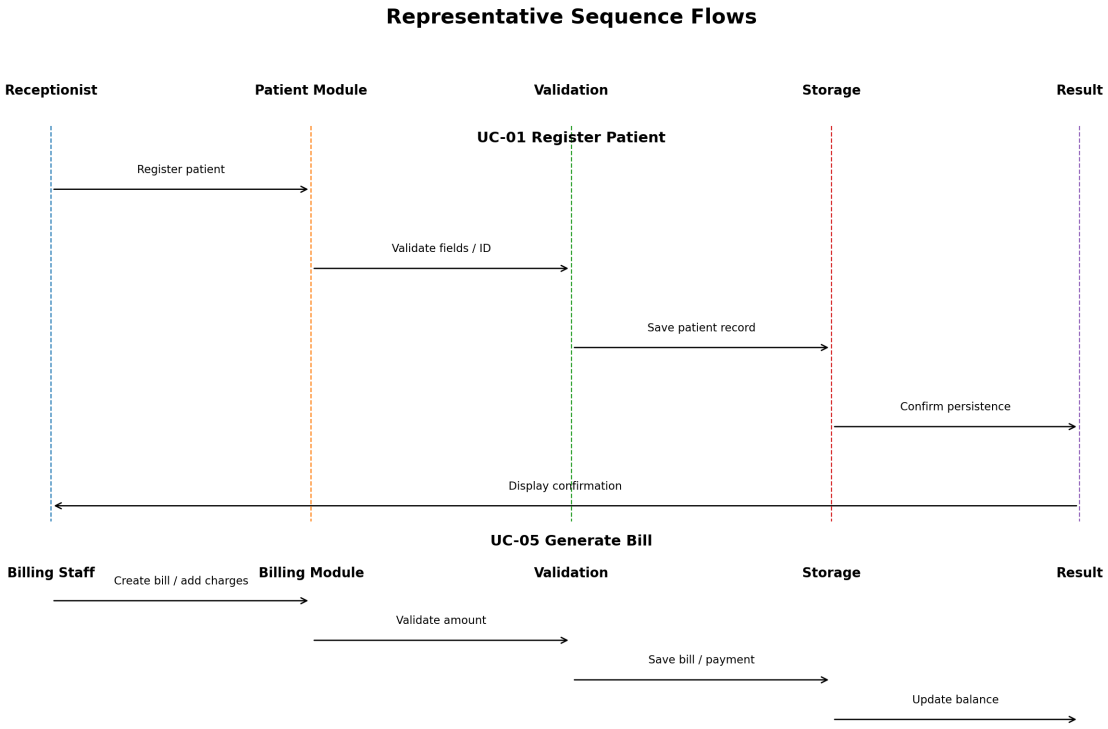


Figure 2. Representative sequence flows for patient registration and billing.

4.3 Module Interface Design

The HMS has no required external REST API in the initial release. The following are proposed internal module interfaces based on the SRS's suggested C/C++ program structure; exact signatures remain an implementation decision.

Component	Representative Interface	Purpose
Patient	registerPatient(), searchPatient(), updatePatient()	Create, retrieve, and update patient records.
Doctor	addDoctor(), updateDoctor(), searchDoctor()	Manage doctor identity and availability.
Appointment	createAppointment(), modifyAppointment(), cancelAppointment()	Schedule and maintain appointments.
Admission/Room	admitPatient(), allocateRoom(), dischargePatient()	Manage admissions, room allocation, and discharge.
Medical Record	addMedicalRecord(), getMedicalHistory()	Record and retrieve clinical information.
Billing	createBill(), recordPayment(), calculateBalance()	Manage charges, payments, and outstanding balances.
Storage	saveRecords(), loadRecords(), backupData()	Persist, reload, and copy important local data.

4.4 Error Handling, Logging & Monitoring

• Invalid, empty, duplicate, or boundary inputs should be rejected with clear corrective messages.

• Invalid identifiers and unavailable rooms/beds should prevent the operation rather than creating inconsistent records.

• Unauthorized operations should be denied without exposing restricted information.

• The application should not terminate unexpectedly because of invalid user input.

• Persistent-file failures should be handled safely to minimize data corruption and loss.

• Quality monitoring should track test execution status, requirement coverage, defects, pass rate, regression/retest completion, and related test metrics.

4.5 UX Design

The initial UI may be console-based. Menus and prompts should be clear and consistent, frequently used operations should require a reasonable number of steps, meaningful error messages should identify invalid input, and successful create/update/delete operations should show confirmation. The main menu is organized around Login, Patient Management, Doctor Management, Appointment Management, Admission and Room Management, Medical Records, Billing and Payments, Reports, Backup/Data Management, and Exit.

4.6 Open Issues & Next Steps

• Database integration using MySQL, PostgreSQL, SQLite, or another DBMS.

• Graphical user interface and web/mobile front end.

• Online appointment booking and patient self-service.

• SMS/email appointment reminders.

• Laboratory, pharmacy, and insurance modules.

• Stronger authentication and role-based audit logs.

• Cloud deployment and multi-user network access.

• Hospital statistics dashboard and analytics.

• Integration with external healthcare interoperability standards.

5. Appendices

5.1 References

• Hospital Management System Software Requirements Specification, Team 1, C/C++ SE Mini Project.

• Hospital Management System Test Plan, Team 1.

• SAD Template supplied with the project.

5.2 Tools

C/C++ compiler and compatible IDE/editor; local file-system storage; command-line or shell scripts for repeatable testing; debugger or memory checker where available; issue/defect tracking; and diagramming tools such as draw.io or PlantUML for maintaining UML views.